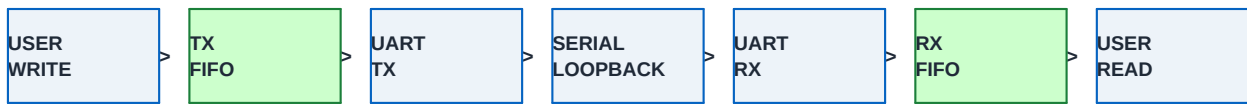


PROJECT WORK SUMMARY

UART Transceiver with TX and RX FIFOs

Completed work, use cases, signal catalog, verification evidence, current limitations, and the recommended next engineering steps.



CURRENT RESULT	IMPLEMENTED	VERIFICATION
12 / 12 PASS	8-N-1 + dual FIFOs	Vivado XSim
Final banner at 4098 ns	16-byte TX and RX queues	Self-checking testbench

Prepared from the current RTL, testbench, Vivado logs, waveform assets, and project documentation in the repository.

Report date: 12 July 2026

1. Executive Summary

The project now implements a complete modular UART loopback datapath in pure Verilog. It combines a baud-rate pulse generator, an 8-N-1 transmitter, a 16x-oversampling receiver, and separate 16-entry byte FIFOs on both the transmit and receive sides. The FIFOs decouple fast user-side activity from the much slower serial line and preserve byte order when writes arrive in bursts or reads are delayed.

What was completed

The top-level receive path was upgraded with an RX FIFO while preserving the existing user interface. The original simple simulation was expanded into a self-checking 12-case suite. A reusable standalone FIFO test instance was added for capacity, simultaneous access, and pointer reuse. Vivado XSim project artifacts, waveform evidence, technical documentation, and interview-ready explanations were also produced.

The key engineering outcomes are:

- Burst tolerance: the TX FIFO accepts multiple bytes while the transmitter is busy.
- Delayed-read tolerance: the RX FIFO holds multiple completed bytes until software pops them.
- Ordered delivery: show-ahead FIFO data exposes the oldest unread byte.
- Clean recovery: reset flushes both queues and both UART state machines, then communication resumes.
- Reusable verification: helper tasks, error counting, a timeout, and a final pass/fail banner make regressions obvious.

Implementation inventory

Block	File	Responsibility
Baud generator	rtlCode/ baud_rate_generator.v	Produces one-cycle TX bit ticks and 16x RX sampling ticks.
Transmitter	rtlCode/uart_sender.v	Frames and serializes bytes as start + 8 LSB-first data bits + stop.
Receiver	rtlCode/uart_receiver.v	Detects start, oversamples, reconstructs 8 bits, validates stop high.
FIFO	rtlCode/uart_fifo.v	16 x 8 synchronous show-ahead queue used once for TX and once for RX.
Integration	rtlCode/uart_top.v	Connects timing, TX FIFO, TX, internal loopback, RX, and RX FIFO.
Verification	rtlCode/uart_top_tb.v	Runs 12 self-checking end-to-end and FIFO unit test cases.

2. Implemented Use Cases

These are the practical behaviors already represented by the RTL and verified by the current testbench.

Use case	How it works	Evidence
Single-byte loopback	A user writes one byte; it is queued, serialized, received, queued again, and exposed at data_out.	TC02
Bit-pattern extremes	00, FF, 55, and AA stress long steady levels and alternating transitions.	TC03-TC05
Back-to-back producer burst	Six writes are issued without waiting for the serial line. The TX FIFO drains them one frame at a time.	TC06
Slow or delayed consumer	Six received bytes remain buffered while rdy_clr is withheld, then pop in original order.	TC07
Reset during activity	An active frame and queued byte are discarded; state, queues, and serial line return to idle.	TC08
Recovery after reset	A new byte transfers correctly after the interrupted transaction reset.	TC09
FIFO full protection	A 17th write is ignored when occupancy reaches 16, preventing memory overwrite.	TC10
Same-cycle read and write	The FIFO pops the old front and writes a new entry while count remains constant.	TC11
Pointer reuse after wrap	The queue accepts and returns another byte after a complete 16-location cycle.	TC12

Current deployment model

The serial line is internal: uart_sender.tx is directly connected to uart_receiver.rx inside uart_top. This is excellent for deterministic simulation and demonstration, but external RX/TX pins and asynchronous-input hardening are still future work.

3. Signal Catalog

The catalog is grouped by visibility and purpose. Public ports are the present integration contract; internal signals explain how bytes move through the datapath.

3.1 uart_top public interface

Signal	Dir / width	Meaning	When it is active
clk	Input / 1	Reference clock for all synchronous logic.	Every rising edge.
reset	Input / 1	Synchronous active-high reset for divider, FSMs, and FIFOs.	High on a rising clk edge.
data_in	Input / 8	Parallel byte presented to the TX FIFO.	Sampled when wr_en is accepted.
wr_en	Input / 1	Requests a TX FIFO enqueue.	High for a clk cycle; ignored if FIFO is full.
rdy_clr	Input / 1	Consumes one byte from the RX FIFO.	High for a clk cycle while rdy is high.
rdy	Output / 1	At least one unread byte exists in the RX FIFO.	High while RX FIFO is non-empty.
busy	Output / 1	UART transmitter FSM is not idle.	High during START, DATA, and STOP states.
data_out	Output / 8	Oldest unread RX FIFO byte (show-ahead data).	Meaningful when rdy is high.

3.2 Timing and serial datapath

Signal	Owner	Meaning
tx_clock_enable	uart_top / BRG	One-clock tick that advances one UART TX bit period.
rx_en	uart_top / BRG	Receiver sampling tick, configured at 16x the UART bit rate.
tx_line	uart_top	Internal serial wire from the transmitter output to the receiver input.
tx_counter	baud_rate_generator	Counts reference clocks up to TX_DIV - 1.
rx_counter	baud_rate_generator	Counts reference clocks up to RX_DIV - 1.

3.3 TX control and queue handshake

Signal	Meaning
tx_fifo_empty	Indicates that no user bytes remain queued for transmission.
tx_fifo_dout	Show-ahead byte at the head of the transmit FIFO.
tx_busy	Internal sender busy output; also drives top-level busy.
tx_start	Combinational pop/start request: !tx_fifo_empty && !tx_busy.

3. Signal Catalog - Internal Control

3.4 RX completion and queue handshake

Signal	Meaning
rx_byte_ready	Receiver FSM has completed a byte and is holding it for capture.
rx_byte	Completed byte from uart_receiver before FIFO storage.
rx_fifo_full	RX queue occupancy is 16; no new byte can be accepted.
rx_fifo_empty	RX queue occupancy is zero; drives rdy low.
rx_fifo_dout	Oldest unread receive byte; drives data_out.
rx_fifo_write	Capture handshake: rx_byte_ready && !rx_fifo_full.
rx_fifo_read	Pop handshake: rdy_clr && !rx_fifo_empty.

3.5 Reusable FIFO internals

Signal	Width	Meaning
mem	16 x 8	Storage array for queued bytes.
wr_ptr	4	Address of the next accepted write; wraps naturally after 15.
rd_ptr	4	Address of the current front entry; wraps naturally after 15.
count	5	Occupancy from 0 through 16.
full	1	count == 16.
empty	1	count == 0.
rd_data	8	Combinational show-ahead value mem[rd_ptr].
do_write	1	wr_en && !full; qualifies storage and pointer update.
do_read	1	rd_en && !empty; qualifies read pointer update.

3.6 UART FSM storage and state

Module	Signal	Purpose
uart_sender	state	IDLE, START, DATA, or STOP.
uart_sender	data	Latched transmit byte so data_in can change safely.
uart_sender	index	Selects data bits 0 through 7, LSB first.
uart_receiver	state	IDLE, START, DATA, or STOP.
uart_receiver	sample	Counts 16x sampling ticks within a UART bit.
uart_receiver	index	Selects the receive bit position.
uart_receiver	temp	Assembles the incoming byte before completion.

4. Verification Performed

Acceptance result: PASS

Vivado XSim reached UART_FIFO_VERIFICATION_PASS with all 12 test cases passing and error_count equal to zero at 4098 ns. The Verilog compile and elaboration logs show successful analysis and elaboration of the baud generator, FIFO, receiver, sender, top, and testbench.

ID	Test	What was checked	Result
TC01	Reset / idle	busy=0, rdy=0, TX line high, both FIFOs empty.	PASS 10 ns
TC02	Single A5	End-to-end send, receive, and pop of A5.	PASS 350 ns
TC03	All zero	00 survives framing, sampling, and buffering.	PASS 670 ns
TC04	All one	FF survives framing, sampling, and buffering.	PASS 990 ns
TC05	Alternating	55 and AA both return correctly.	PASS 1630 ns
TC06	TX burst	11,22,33,44,77,88 accepted while TX is active; TX FIFO drains.	PASS 3547 ns
TC07	RX hold / order	Six bytes stay queued; front stays stable; ordered pops empty queue.	PASS 3590 ns
TC08	Reset active transfer	FSM stops, queues flush, rdy clears, serial line returns high.	PASS 3606 ns
TC09	Post-reset recovery	3C transfers correctly after reset interruption.	PASS 3946 ns
TC10	FIFO capacity	16 entries fill FIFO; EE overflow write is ignored; 0..15 read back.	PASS 4078 ns
TC11	Simultaneous I/O	Read 5C while writing D2; count stays one; D2 becomes front.	PASS 4090 ns
TC12	Pointer reuse	9B writes and reads after full pointer cycle.	PASS 4098 ns

How the testbench proves correctness

- A fast simulation configuration uses TX_DIV=16 and RX_DIV=1 so full UART frames complete quickly while retaining the 16:1 sampling relationship.
- Dedicated tasks drive reset, enqueue bytes, acknowledge receive data, and check FIFO front values.
- Case inequality is used for byte comparisons so X or Z values cannot silently pass.
- Each failed condition increments error_count; any nonzero total calls \$fatal.
- A separate 20,000 ns watchdog calls \$fatal if the simulation hangs.
- A standalone uart_fifo instance isolates queue edge behavior from the end-to-end UART datapath.

5. Waveform and Artifact Evidence

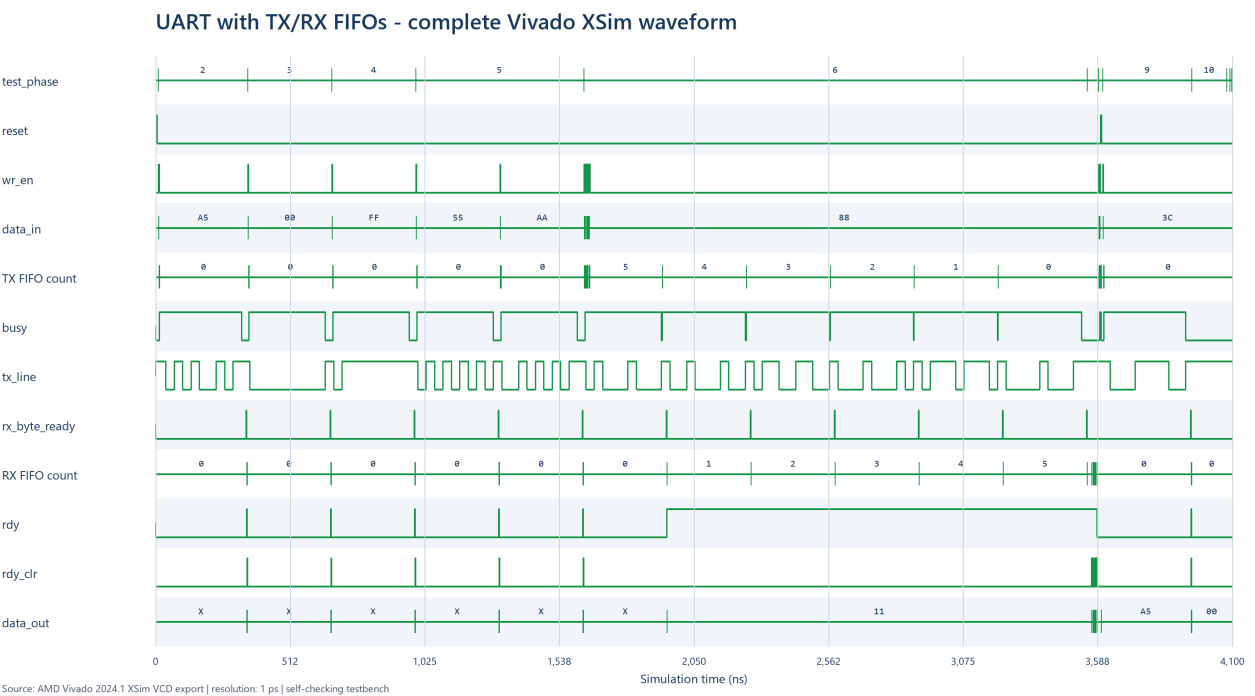


Figure 1. Complete XSim waveform summary from reset through TC12. Signals show TX burst occupancy draining, RX FIFO accumulation, ready/clear handshakes, serialized traffic, and post-reset recovery.

Artifact	Purpose
vivado/uart_fifo_verification/uart_fifo_verification.xpr	Reopenable Vivado project.
vivado/uart_fifo_verification/uart_fifo_waveform.wcfg	Saved waveform signal configuration.
vivado/uart_fifo_verification/uart_fifo_waveform.vcd	Portable simulation value-change data.
.../xsim/simulate.log	Exact 12-case pass transcript and final acceptance banner.
report_assets/uart_xsim_waveform_*.png	Overview and detailed readable waveform images.
outputs/uart_fifo_verification/	Detailed verification report, editable document, and test-case workbook.

What is verified versus what is not

The current evidence is behavioral simulation evidence. It verifies RTL function under the modeled clock, divider, and internal loopback conditions. It is not yet proof of FPGA timing closure, pin behavior, asynchronous RX robustness, electrical compatibility, or sustained operation against an external UART device.

6. Current Limitations and Engineering Risks

Area	Current state	Why it matters
External interface	TX and RX are connected internally; no external serial ports are exposed.	The design cannot yet connect directly to a USB-UART bridge or another board.
Clock-domain safety	No two-flop synchronizer is present on an external RX input.	A real asynchronous pin can cause metastability in the receiver.
Sampling robustness	One sample per data bit after 16x timing; no majority vote.	Noise and clock mismatch tolerance are lower than a hardened UART.
Error reporting	Invalid stop bits suppress rdy, but no framing, parity, break, or overrun status is exported.	Software cannot diagnose why a byte was lost or rejected.
Backpressure	FIFO full flags are internal. A full TX FIFO ignores writes; RX can only hold 16 bytes.	The surrounding system cannot prevent or report data loss under sustained overload.
Configurability	8-N-1 only; divider counter widths are fixed at 13 and 10 bits.	Other clock/ baud combinations may need wider counters and compile-time safeguards.
FIFO reuse	Depth and data width are fixed; implementation is single-clock only.	It cannot directly serve other widths, depths, or asynchronous clock domains.
FPGA readiness	No synthesis, implementation, XDC constraints, timing report, or hardware loopback run is included.	Behavioral success does not establish board-level operation or timing closure.

Important interpretation

The existing design is a strong learning and verification baseline. The next step is not to add features randomly; it is to create a safe external UART boundary, expose flow-control/status information, and then prove the design against realistic asynchronous conditions.

7. Recommended Future Work

The following sequence turns the verified simulation core into a hardware-ready peripheral while keeping each milestone independently testable.

1

Expose a real serial interface

Add tx output and rx input ports, plus an optional LOOPBACK parameter or test-only mux. Keep the internal loopback path available for regression.

Exit condition: External pins exist without breaking the 12-case internal regression.

2

Harden the asynchronous RX boundary

Add a two-flop synchronizer before start detection. Consider a 3-sample majority vote around the bit center and confirm divider tolerance.

Exit condition: Metastability risk is contained and noise/jitter tests pass.

3

Expose status and flow control

Export tx_full, tx_empty, rx_full, rx_empty, and occupancy or almost-full flags. Add sticky overflow, framing, and break indicators with explicit clear behavior.

Exit condition: The host can avoid silent write loss and diagnose receive faults.

4

Expand protocol configurability

Support selectable baud divisors, parity, stop-bit count, and data width. Derive counter widths with $\lceil \log_2 \rceil$ and add parameter validity checks.

Exit condition: Multiple frame formats and clock/ baud combinations regress cleanly.

5

Strengthen verification

Drive RX independently instead of only loopback. Add false starts, bad stop bits, overflow, underflow, jitter, baud mismatch, random traffic, assertions, scoreboarding, and functional coverage.

Exit condition: The regression covers both normal traffic and defined failure behavior.

6

Synthesize and test on FPGA

Choose a target board, add clock and pin constraints, run synthesis/implementation/timing, then test with a USB-UART bridge at real baud rates.

Exit condition: Timing closes and hardware loopback transfers long streams without errors.

7

Integrate as a reusable peripheral

Wrap the UART with AXI4-Lite, APB, or a simpler register interface. Add RX-available and TX-space interrupts; consider DMA only if throughput needs it.

Exit condition: Firmware can configure, transmit, receive, and service interrupts through registers.

8. Your Immediate Next Step

Recommended next sprint: external UART readiness

Start with steps 1 through 3 together: expose tx/rx, synchronize rx, and export FIFO/error status. This is the smallest coherent upgrade because it makes the design physically connectable without hiding overload or asynchronous-input risks.

Order	Action	Definition of done
1	Refactor uart_top ports	External tx and rx are present; internal loopback is optional.
2	Add RX synchronizer	Two registered stages sit before the receive FSM; simulation still passes.
3	Expose FIFO status	The host can see full/empty and cannot unknowingly overrun TX.
4	Add error outputs	Bad stop-bit and overflow tests set documented sticky flags.
5	Build an external-line testbench	Independent RX stimulus covers correct, jittered, and malformed frames.
6	Add XDC and run implementation	Clock constraint is correct; timing closes for the chosen FPGA.
7	Run board loopback	A PC script or terminal transfers a long pattern with zero mismatches.

Suggested verification gates for the next sprint

- Keep all current TC01-TC12 cases as a non-regression suite.
- Add a test where RX start pulses are too short and must be rejected.
- Add +/-2 percent baud mismatch and controlled edge jitter tests.
- Fill both integrated FIFOs and verify the documented full/overflow behavior.
- Run synthesis lint and check CDC warnings before going to the board.

Expected future outcome

After this sprint, the project moves from a verified internal-loopback learning design to a credible FPGA UART IP block with a safe asynchronous input, visible flow control, diagnosable errors, and a clear path to bus integration.

9. Evidence Basis and Handoff

This report was derived from the following repository evidence rather than from assumptions:

Evidence	What it supports
rtlCode/*.v	Implemented architecture, interfaces, state machines, FIFO behavior, and limitations.
rtlCode/uart_top_tb.v	Exact test stimuli, checks, expected values, watchdog, and pass/fail criteria.
vivado/.../xsim/compile.log	Successful Verilog analysis of all design and testbench modules.
vivado/.../xsim/elaborate.log	Successful XSim elaboration of uart_top_tb.
vivado/.../xsim/simulate.log	Exact TC01-TC12 timestamps and final PASS banner at 4098 ns.
report_assets/*.png	Readable waveform views produced from the simulation data.
PROJECT_INTRODUCTION.md and UART_PROJECT_NOTES.md	Architecture intent, system story, and documented operating model.
INTERVIEW_EXPLANATION.md	Presentation-ready summary of the completed design and verification.

Final project status

The current behavioral milestone is complete: dual FIFO integration is implemented and the self-checking regression passes all 12 cases. The next milestone should be external-interface hardening and FPGA validation, following the roadmap in Sections 7 and 8.